Autenticación y Autorización

Introducción al proceso de Autenticación y Autorización con JWT y JAX-RS

Arquitectura del Software

Curso 2025/26

Contenidos

- Autenticación vs Autorización
- Token JWT
- Emisión del token JWT en JAX-RS
- Control de autorización en JAX-RS
- Autorización basada en roles

Autenticación vs Autorización

Autenticación vs autorización:

- Autenticación: proceso que valida que un usuario es quien dice ser.
- Autorización: proceso que concede al usuario permiso para acceder a un recurso o funcionalidad.

Token JWT

JSON Web Token : especificación que permite el intercambio fiable de información entre dos partes.

El token consta de tres apartados:

- **Encabezado**: indica el tipo y el algoritmo de firma

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

Token JWT

- **Cuerpo**: indica el usuario (`sub`) y cualquier otra información

```
{
  "sub": "23242343",
  "exp": 1740052591,
  "name": "Juan",
  "roles": "ADMIN,USER"
}
```

- **Firma**: se calcula utilizando el encabezado y el cuerpo codificado en Base64 y una clave secreta.

Las dos primeras partes codificadas y la firma se combinan para generar el token.

Generación de un token JWT

En la asignatura se utilizará `jjwt` (Java JWT):

```
<dependency>
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt</artifactId>
        <version>0.9.1</version>
</dependency>
```

Generación de un token JWT

```
Map<String, Object> claims = // el cuerpo del token

Date caducidad = Date.from(
    Instant.now()
    .plusSeconds(3600)); // 1 hora de validez

String token = Jwts.builder()
        .setClaims(claims)
        .signWith(SignatureAlgorithm.HS256, SECRET_KEY)
        .setExpiration(caducidad)
        .compact();
```

- En los *claims* debe incluirse "sub" (Subject) con el identificador del usuario. Adicionalmente se pueden introducir otras propiedades.

Comprobación validez token JWT

```
Claims claims = Jwts.parser()
                    .setSigningKey(SECRET_KEY)
                    .parseClaimsJws(token)
                    .getBody();
```

- Si el token no es válido (formato o firma no es correcta), se notifica una excepción.
- En caso de que esté caducado, también se notifica un error.

Emisión del token JWT

El token se emite una vez que el usuario se ha autenticado:

- En el siguiente ejemplo se utiliza autenticación mediante usuario/contraseña.
- En un controlador con ruta "auth", se registra la operación "login":

```
@POST
@Path("/login")
public Response login(
    @FormParam("username") String username,
    @FormParam("password") String password) {

// Verificar las credenciales y emisión del token

}
```

Emisión del token JWT

```
// Verificar las credenciales y emisión del token

Map<String, Object> claims = verificarCredenciales(username, password);
if (claims != null) {
    String token = // generar el token
    return Response.ok(token).build();
} else {
    return Response
            .status(Response.Status.UNAUTHORIZED)
            .entity("Credenciales inválidas").build();
}
```

Solicitud de un token JWT

Utilizando el comando cURL:

```
curl -X POST -H "Content-Type: application/x-www-form-urlencoded"
    -d "username=juan&password=clave"
    http://localhost:8080/api/auth/login
```

- La respuesta del comando es el token.

Envío del token en una solicitud

El token se envía en la cabecera *Authorization* de tipo "Bearer".

```
curl -X GET -H "Authorization: Bearer token_jwt"
  http://localhost:8080/api/actividades/1
```

Control de autorización

La aplicación debe comprobar en cada petición a un recurso protegido que el usuario tiene un **token válido**.

En JAX-RS se implementa con un **filtro**:

```
@Provider
@Priority(Priorities.AUTHENTICATION)
public class JwtTokenFilter implements ContainerRequestFilter {

    @Override
    public void filter(ContainerRequestContext requestContext)  {

        // Implementación del control de autorización
    }
}
```

Control de autorización: filtro

El filtro detiene la petición con el método `abortWith`.

```
// Implementación del control de autorización
String authorization = requestContext.getHeaderString("Authorization");

if (authorization == null || !authorization.startsWith("Bearer ")) {
    requestContext.abortWith(
        Response.status(Response.Status.UNAUTHORIZED).build());
} else {
    String token = authorization.substring("Bearer ".length()).trim();
    try {
        // Validar el token ...
    } catch (Exception e) { // Error de validación
        requestContext.abortWith(
            Response.status(Response.Status.UNAUTHORIZED).build());
    }
}
```

Control de autorización: rutas públicas

El filtro debe ser configurado con las rutas que sean públicas:

```
public void filter(ContainerRequestContext requestContext)  {

        String path = requestContext.getUriInfo().getPath();

        // rutas públicas
        if (path.equals("auth/login") || path.equals("/otra")) {
            return; // no se controla la autorización
        }
        // ...
}
```

- Más adelante se presenta una alternativa basada en anotaciones.

Autorización con roles

En JAX-RS se ofrece la anotación `RolesAllowed` para indicar los roles de usuario que pueden ejecutar una operación:

```
    @POST
    @Consumes(MediaType.APPLICATION_XML)
    @RolesAllowed("PROFESOR")
    public Response createActividad(Actividad actividad) throws Exception
```

Autorización con roles

En la **emisión del token** JWT debe incluirse una propiedad en los *claims* (por ejemplo, "roles") para incluir los **roles** del usuario autenticado en la aplicación.

El filtro debe controlar que el token del usuario tenga alguno de los roles permitidos (anotación)

- Para procesar la anotación `@RolesAllowed`, el filtro requiere que sea inyectada la información del recurso que se ha solicitado:

```
@Context
private ResourceInfo resourceInfo;
```

Autorización con roles: control en el filtro

```
Set<String> roles = new HashSet<>(
Arrays.asList(claims.get("roles", String.class).split(",")));

// Consulta si la operación está protegida por rol
if (this.resourceInfo.getResourceMethod()
    .isAnnotationPresent(RolesAllowed.class)) {

    String[] allowedRoles = resourceInfo.getResourceMethod()
        .getAnnotation(RolesAllowed.class).value();

    if (roles.stream()
        .noneMatch(userRole -> Arrays.asList(allowedRoles)
                        .contains(userRole))) {
        requestContext.abortWith(
            Response.status(Response.Status.FORBIDDEN).build());
    }
}
```

Rutas públicas

El uso del objeto `ResourceInfo` permite controlar las **rutas públicas** con la anotación `@PermitAll`.

Al comienzo del filtro se incluiría:

```
// Comprobamos si la ruta tiene la anotación @PermitAll
if (resourceInfo.getResourceMethod()
        .isAnnotationPresent(PermitAll.class)) {
    return;
}
```

- Esta opción evita controlar las rutas con valores fijos.

Disponibilidad de la información del token

Es útil que la información del token esté disponible en el controlador REST.

Existen dos modos de establecer la información:

1. En el contexto de seguridad de JAX-RS: disponible para la aplicación, pero limitado en información.
2. En la petición HTTP: más flexible, pero limitada a la petición.

Es más común utilizar la opción 2.

Disponibilidad de la información del token

Opción 2: Añadir la información a la petición

- En el filtro, inyectamos la petición HTTP y añadimos los *claims*:

```
    @Context
    private HttpServletRequest servletRequest;
```

```
    this.servletRequest.setAttribute("claims", claims);
```

En `pom.xml` se añade la dependencia:

```
<dependency>
    <groupId>javax.servlet</groupId>
    <artifactId>servlet-api</artifactId>
    <version>2.5</version><scope>provided</scope>
</dependency>
```

Disponibilidad de la información del token

- En el controlador REST, también inyectamos la petición HTTP y se extrae la información:

```
if (this.servletRequest.getAttribute("claims") != null) {

    Claims claims = (Claims) this.servletRequest.getAttribute("claims");
    System.out.println("Usuario autenticado: " + claims.getSubject());
    System.out.println("Roles: " + claims.get("roles"));
}
```